

PROGRAMMING IN C



Pointers and dynamic memory allocation and C pre-processor

-
- **5.1** Introduction to pointers: Pointer declaration and initialization, Pointer addition and subtraction, Evaluating pointer expressions Pointers and Functions: Pass by Reference, Returning pointers from functions, File Handling
- **5.2** Dynamic Memory Allocation using Pointers: Dynamic memory allocation using malloc(), calloc() and realloc() and deallocation of memory using free()
- **5.3** C Pre-processor, Directives, Macros

4.3

Introduction to pointers: Pointer declaration and initialization,
Pointer addition and subtraction, evaluating pointer expressions
Pointers and Functions: Pass by Reference, Returning pointers
from functions

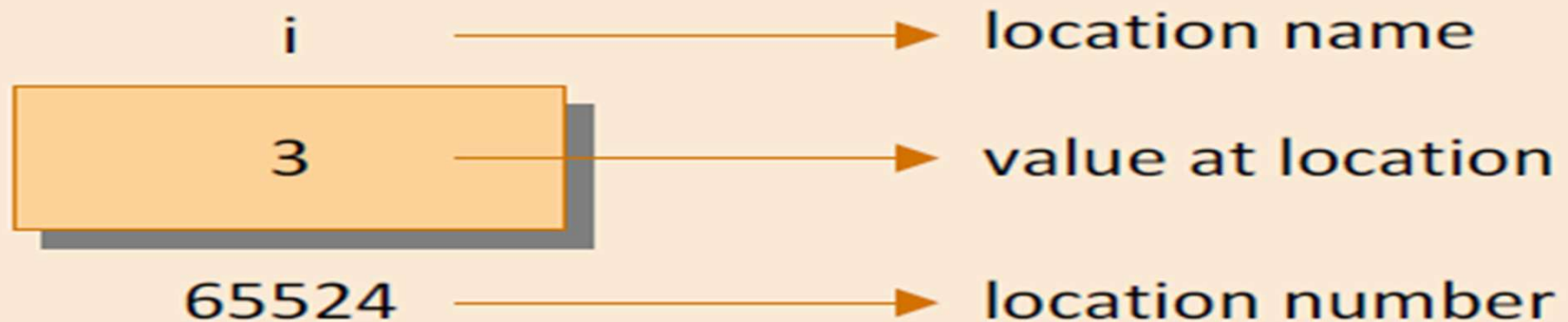
Introduction

Consider the declaration,

```
int i = 3 ;
```

This declaration tells the C compiler to:

- (a) Reserve space in memory to hold the integer value.
- (b) Associate the name **i** with this memory location.
- (c) Store the value 3 at this location.



What is pointer?

- Pointers are nothing but memory address
- Pointer is a variable that represents the location of data item
- Pointers can be used for variety of applications such as to create complex data structures, trees , linked lists, linked stacks, linked queues, and graphs
- For dynamic allocation of memory

How to use pointers?

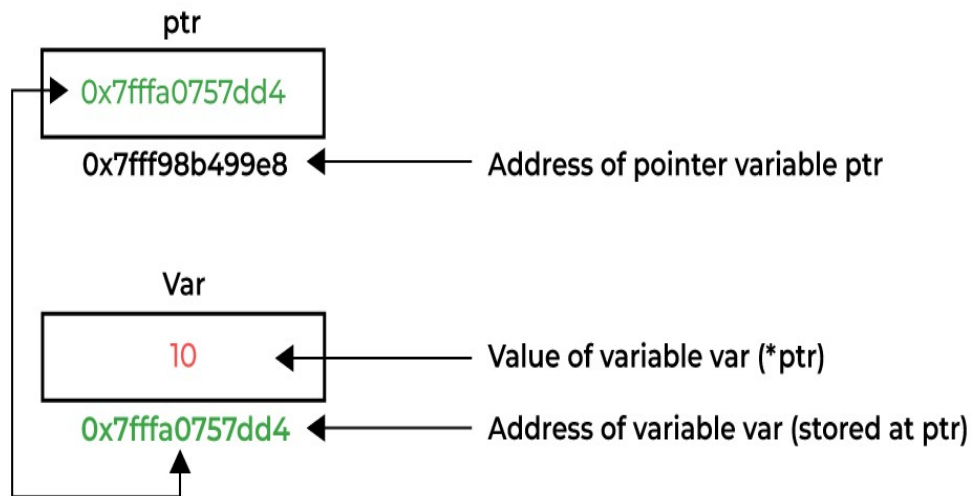
- ✓ Pointer declaration
- ✓ Pointer initialization
- ✓ Pointer Dereferencing

Pointer Declaration & initialization

- In pointer declaration, we only declare the pointer but do not initialize it.
- To declare a pointer, we use the (*) **dereference operator** before its name.
- `int *ptr;`
- Pointer initialization is the process where we assign some initial value to the pointer variable.
- We generally use the (&) **address of operator** to get the memory address of a variable and then store it in the pointer variable.
- `int x = 100;`
`int * ptr;`
`ptr = &x;`

Pointer Dereferencing

- Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same (*****) **dereferencing operator** that we used in the pointer declaration.



Recap of Pointer Terms

Term	Meaning
ptr	pointer (holds address)
&num	address of num
*ptr	Value at that address

Example:1

```
void myfun()  
{  
    int x = 10;  
    // declare pointer variable  
    int* ptr;  
    // note that data type of ptr and var must be same  
    ptr = &x;  
    // assign the address of a variable to a pointer  
    printf("Value at ptr = %p \n", ptr);  
    printf("Value at var = %d \n", x);  
    printf("Value at *ptr = %d \n", *ptr);  
}  
  
// Driver program  
int main()  
{  
    myfun();  
    return 0;  
}
```

```
Value at ptr = 000000000061FDE4  
Value at var = 10  
Value at *ptr = 10
```

Pointer arithmetic

- increment/Decrement of a Pointer
- Addition of integer to a pointer
- Subtraction of integer to a pointer
- Subtracting two pointers of the same type
- Comparison of pointers

Addition of Integer to Pointer

- When a pointer is added with an integer value, the value is first multiplied by the size of the data type and then added to the pointer.
- **Example:**
- Consider the same example as above where the **ptr** is an **integer pointer** that stores **1000** as an address. If we add integer 5 to it using the expression, **ptr = ptr + 5**, then, the final address stored in the ptr will be **ptr = 1000 + sizeof(int) * 5 = 1020**.

Subtraction of Integer to Pointer

- When a pointer is subtracted with an integer value, the value is first multiplied by the size of the data type and then subtracted from the pointer similar to addition.

Example

- Write a program in C using pointer which takes an input from the user and then checks whether it is a number or a character. If it is a character, determine whether it is in upper case or lower case.
- `#include <ctype.h>` -

Function	Purpose
<code>isdigit()</code>	Checks if the given character is a digit (0-9).
<code>isalpha()</code>	Checks if the given character is an alphabet letter (A-Z or a-z).
<code>isupper()</code>	Checks if the given character is an uppercase letter (A-Z).
<code>islower()</code>	Checks if the given character is a lowercase letter (a-z).

Changing Value Using Pointer

```
#include <stdio.h>
```

```
int main() {  
    int num = 50;  
    int *ptr;  
    ptr = &num;
```

```
    printf("Before: num = %d\n", num);
```

```
    *ptr = 100; // Change value using pointer!
```

```
    printf("After: num = %d\n", num);
```

```
    return 0;
```

```
}
```

Swapping Two Numbers (Using Pointers)

```
#include <stdio.h>
```

```
void swap(int *x, int *y) {  
    int temp = *x; // save x's value  
    *x = *y;       // put y in x  
    *y = temp;     // put temp in y  
}
```

```
int main() {  
    int a = 5, b = 10;  
  
    printf("Before Swap: a = %d, b = %d\n", a, b);  
  
    swap(&a, &b); // send addresses!  
  
    printf("After Swap: a = %d, b = %d\n", a, b);  
  
    return 0;  
}
```


Pointer with array

```
#include <stdio.h>
```

```
int main() {  
    int arr[5] = {10, 20, 30, 40, 50};  
    int *ptr = arr; // points to arr[0]  
  
    for (int i = 0; i < 5; i++) {  
        printf("arr[%d] = %d\n", i, *(ptr + i));  
    }  
  
    return 0;  
}
```

Pointer to pointer

- A **pointer to a pointer** in C is exactly what it sounds like: it's a pointer that stores the address of another pointer!
- **Let's break it down:**
 - A **pointer** is a variable that holds the **address** of another variable.
 - A **pointer to a pointer** holds the **address of another pointer**, which in turn holds the address of a variable.

Pointer to pointer

```
#include <stdio.h>
```

```
int main() {  
    int num = 25;  
    int *ptr = &num;  
    int **pptr = &ptr; // pointer to pointer!  
  
    printf("num = %d\n", num);  
    printf("*ptr = %d\n", *ptr);  
    printf("**pptr = %d\n", **pptr);  
  
    return 0;  
}
```

Function Calls

The two types of function calls—call by value and call by reference.

Arguments can generally be passed to functions in one of the two ways:

(a) sending the values of the arguments : ‘value’ of each of the actual arguments in the calling function is copied into corresponding formal arguments of the called function.

With this method, the changes made to the formal arguments in the called function have no effect on the values of actual arguments in the calling function.

(b) sending the addresses of the arguments

Function Calls: Sending the values of the arguments

```
# include <stdio.h>
void swapv ( int x, int y ) ;
int main( )
{
int a = 10, b = 20 ;
swapv ( a, b ) ;
printf ( "a = %d b = %d\n", a, b ) ;
return 0 ;
}
void swapv ( int x, int y )
{
int t ;
t = x ;
x = y ;
y = t ;
printf ( "x = %d y = %d\n", x, y ) ;
}
```

Output

x = 20 y = 10

a = 10 b = 20

Pointer with function

```
#include <stdio.h>
```

```
void changeValue(int *ptr) {  
    *ptr = 20; // Changing the value at the address  
}
```

```
int main() {  
    int num = 10;  
    printf("Before: %d\n", num);  
  
    changeValue(&num); // Passing address of num  
  
    printf("After: %d\n", num);  
    return 0;  
}
```

Pointer with Strings

```
#include <stdio.h>
```

```
int main() {
```

```
    char *str = "Hello, World!";
```

```
    printf("%s\n", str);
```

```
    // Print each character using pointer
```

Call by reference

The addresses of actual arguments in the calling function are copied into the formal arguments of the called function.

This means that, using these addresses, we would have an access to the actual arguments and hence we would be able to manipulate them.

Call by reference

```
# include <stdio.h>
```

```
void swapr ( int *, int * ) ;
```

```
void swapr ( int *x, int *y )
```

```
{ int t ;
```

```
    t = *x ;
```

```
    *x = *y ;
```

```
    *y = t ;
```

```
}
```

```
int main( )
```

```
{ int a = 10, b = 20 ;
```

```
    swapr ( &a, &b ) ;
```

```
    printf ( "a = %d b = %d\n", a, b ) ;
```

Output

a = 20 b = 10

Call by reference

- Using a call by reference intelligently, we can make a function return more than one value at a time, which is not possible ordinarily.

Call by reference

```
# include <stdio.h>
void areaperi ( int, float *, float * ) ;
int main( )
{
    int radius ;
    float area, perimeter ;
    printf ( "Enter radius of a circle " ) ;
    scanf ( "%d", &radius ) ;
    areaperi ( radius, &area, &perimeter ) ;
    printf ( "Area = %f\n", area ) ;
    printf ( "Perimeter = %f\n", perimeter ) ;
    return 0 ;
}
```

Call by reference

```
void areaperi(int, float *, float *);  
int main()  
{  
    int radius;  
    float area, perimeter;  
    printf("Enter radius of a circle: ");  
    scanf("%d", &radius);  
    areaperi(radius, &area, &perimeter);  
    printf("Area = %.2f\n", area);  
    printf("Perimeter = %.2f\n", perimeter);  
    return 0;  
}  
void areaperi(int r, float *a, float *p)  
{  
    *a = 3.14f * r * r;  
    *p = 2 * 3.14f * r;  
}
```

Output:

Enter radius of a circle 5
Area = 78.5
Perimeter = 31.4

Summary

- (a) Pointers are variables which hold addresses of other variables.
- (b) A pointer to a pointer is a variable that holds address of a pointer variable.
- (c) The & operator fetches the address of the variable in memory.
- (d) The * operator lets us access the value present at an address in memory with an intention of reading it or modifying it.
- (e) A function can be called either by value or by reference.
- (f) Pointers can be used to make a function return more than one value simultaneously in an indirect manner.

What will be the output ?

```
# include <stdio.h>
void fun ( int, int ) ;
int main( )
{
    int i = 5, j = 2 ;
    fun ( i, j ) ;
    printf ( "%d %d\n", i, j ) ;
    return 0 ;
}
void fun ( int i, int j )
{
    i = i * i ;
    j = j * j ;
}
```

What will be the output ?

```
# include <stdio.h>
void fun ( int *, int * ) ;
int main( )
{
    int i = 5, j = 2 ;
    fun ( &i, &j ) ;
    printf ( "%d %d\n", i, j ) ;
    return 0 ;
}
void fun ( int *i, int *j )
{
    *i = *i * *i ;
    *j = *j * *j ;
}
```

Dynamic memory allocation

In **C**, **dynamic memory allocation** refers to allocating memory during **runtime** instead of at **compile time**.

It allows the program to request memory space while the program is running.

Why use Dynamic Memory Allocation?

We may not know the exact memory requirements during compile time.

More **flexibility**: allocate memory as needed, free it when no longer required.

Useful in **data structures** like linked lists, trees, etc.

Need of Dynamic Memory Allocation

As we know, the size of an array in C is fixed and should be known at compile time. There can be two problems:

The size of the array is not sufficient to store all the elements. To resolve this, one might set the size to store the maximum theoretically possible elements. This creates another problem.

This size of the array is much more than what is required to store the elements. This leads to the wastage of memory.

This is where the dynamic memory allocation comes in.

The size of the array can be increased if more elements are to be inserted and decreased if less elements are inserted.

Moreover, there is no need to estimate the max possible size.

The size can be decided at runtime according to the requirement.

Dynamic Memory Allocation

Dynamic memory allocation is possible in C by using 4 library functions provided by **<stdlib.h>** library:

- malloc()
- calloc()
- free()
- realloc()

Dynamic memory allocation

malloc() Function in C

- malloc() stands for **memory allocation**.
- It dynamically allocates a **block of memory** in the **heap** and returns a **pointer** to the **first byte** of that block.

ptr = (cast_type *) malloc(size_in_bytes);

If the allocation **fails**, malloc() returns NULL.

Steps of Using malloc()

Include the header: `#include <stdlib.h>`

Declare a pointer of appropriate type.

Use malloc() to allocate memory.

Check if memory allocation was successful.

Use the allocated memory.

Free the memory when it's no longer needed using free()

malloc() function can be used to allocate space in memory on the fly during execution of the program

Example

- User should enter 6 names (strings).
- Allocate exact memory for each name dynamically using `malloc()`.
- Store the names in an array of character pointers.
- Display all the entered names.

```
int main( )
{
char *names[ 6 ] ;
char n[ 50 ] ; //Temporary array n is used to store the name input from the user (maximum 49 characters + \0)//
int len, i ;
char *p ;
for ( i = 0 ; i <= 5 ; i++ ) //A for loop runs 6 times (i from 0 to 5) to accept 6 names.//
{
    printf ( "Enter name " ) ;
    scanf ( "%s", n ) ;
    len = strlen ( n ) ; //Gets the length of the entered name (without including the \0)//
    p = ( char * ) malloc ( len + 1 ) ; //* +1 for accommodating \0 *//
    strcpy ( p, n ) ; //Copies the name stored in n to the newly allocated memory (p).//
    names[ i ] = p ; //Stores the pointer p (which points to the copied name) in the names array.
                        Now names[i] points to the dynamically allocated memory holding that name.//
}
for ( i = 0 ; i <= 5 ; i++ )
    printf ( "%s\n", names[ i ] ) ;
return 0 ;
}
```

malloc()

This function requires the number of bytes to be allocated and returns the base address of the chunk of memory that it allocates. The address returned by this function is always of the type **void ***. This is because **malloc()** doesn't know what for did we allocate the memory.

A **void *** means a pointer which is a legal address but it is not address of a **char**, or address of an **int**, or address of any other datatype. Hence it has been converted into **char *** using a C language feature called typecasting.

The prototype of this function has been declared in the header file 'stdlib.h'. Hence we have **#included** this file.

malloc()

But why did we not use array to allocate memory?

This is because, with arrays, we have to commit to the size of the array at the time of writing the program. Moreover, there is no way to increase or decrease the array size during execution of the program.

In other words, when we use arrays, static memory allocation takes place. Unlike this, using **malloc()**, we can allocate memory dynamically, during execution. The argument that we pass to **malloc()** can be a variable whose value can change during execution.

Once we have allocated the memory using **malloc()**, we have copied the name received through the keyboard into this allocated space and finally stored the address of the allocated chunk in the appropriate element of **names[]**, the array of pointers to strings.


```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr; // Pointer to int
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    // Dynamically allocate memory using malloc()
    ptr = (int*) malloc(n * sizeof(int));

    // Check if memory has been allocated successfully
    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        return 1; // Exit program
    }

    // Memory has been successfully allocated
    printf("Memory successfully allocated using malloc().\n");

    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d ", ptr[i]);
    }

    // Free the memory
    free(ptr);
    printf("\nMemory successfully freed.\n");

    return 0;
}

```

Dynamic memory allocation

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    ptr = (int *)malloc(n * sizeof(int)); // allocate memory for n integers

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        return 1;
    }

    // Assign values
    for (int i = 0; i < n; i++) {
        ptr[i] = i + 1;
    }

    // Print values
    for (int i = 0; i < n; i++) {
        printf("%d ", ptr[i]);
    }

    free(ptr); // free memory!

    return 0;
}
```

Dynamic memory allocation

```
#include <stdio.h>
#include <stdlib.h> // Required for malloc

int main() {
    int *ptr;
    ptr = (int*) malloc(sizeof(int)); // Allocate memory

    if (ptr == NULL) {
        printf("Memory allocation failed!");
        return 1;
    }

    *ptr = 42; // Store value in allocated memory
    printf("Dynamically allocated value: %d\n", *ptr);

    free(ptr); // Free memory
    return 0;
}
```

Function returning Pointer

```
#include <stdio.h>
```

```
int* getMax(int *a, int *b) {  
    if (*a > *b)  
        return a;  
    else  
        return b;  
}
```

```
int main() {  
    int x = 15, y = 30;  
    int *maxPtr = getMax(&x, &y);  
  
    printf("Greater number is: %d\n", *maxPtr);  
  
    return 0;  
}
```

Null pointers

- It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned NULL is called a null pointer.

```
#include <stdio.h>
int main ()
{
    int *ptr = NULL;
    printf("The value of ptr is : %0x\n", ptr );
    return 0;
}
```

Introduction to File Handling

What is file handling?

Why use files in C programming?

Types of files: Text files and Binary files

Why use files in C programming?

Files are essential for storing and managing data efficiently in programming.

- **Persistent Storage** – Data stored in files remains intact even after the program terminates.
- **Efficient Handling of Large Data** – Files allow manipulation of large amounts of information without exhausting memory.
- **Data Sharing & Interoperability** – They enable different programs or systems to access and modify shared data.
- **Logging & Backup** – Files help maintain logs, records, and backups to prevent data loss.
- **Structured Storage** – They facilitate organized storage in formats like text, binary, CSV, or databases.

File Handling Functions in C

`fopen()`: Opens a file

`fclose()`: Closes a file

`fprintf()` and `fscanf()`: Read/write to a file (text mode)

`fwrite()` and `fread()`: Read/write to a file (binary mode)

`fseek()`, `ftell()`, `rewind()`: File positioning functions

File Opening Modes

Mode	Description
"r"	Open for reading
"w"	Open for writing (overwrite)
"a"	Append to file
"r+"	Read and write
"w+"	Read and write (overwrite)
"a+"	Read and append

Note that "r" is a string and not a character; hence the double quotes and not single quotes.

Opening and Closing a File

```
#include <stdio.h>

int main()
{
    FILE *fp;
    fp = fopen("example.txt", "w");
    if (fp == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }
    fclose(fp);
    return 0;
}
```

Writing to a File (Text Mode)

```
#include <stdio.h>

int main()
{
    FILE *fp = fopen("example.txt", "w");
    if (fp == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }
    fprintf(fp, "Hello, File Handling in C!\n");
    fclose(fp);
    return 0;
}
```

To be able to successfully read from a file information like mode of opening, size of file, place in the file from where the next read operation would be performed, etc., has to be maintained. Since all this information is inter-related, all of it is gathered together by **fopen()** in a structure called **FILE**.

The **FILE** structure has been defined in the header file “stdio.h”

fopen() returns the address of this structure, which we have collected in the structure pointer called **fp**. We have declared **fp** as follows:
FILE *fp ;

Writing and Reading in Binary Mode

Example of fwrite() and fread()

```
#include <stdio.h>
struct Data
{   int id;
    char name[20];
};
int main()
{   struct Data d1 = {1, "Alice"};
    FILE *fp = fopen("data.bin", "wb");
    fwrite(&d1, sizeof(struct Data), 1, fp);
    fclose(fp);
    struct Data d2;
    fp = fopen("data.bin", "rb");
    fread(&d2, sizeof(struct Data), 1, fp);
    printf("ID: %d, Name: %s\n", d2.id, d2.name);
    fclose(fp);
    return 0;
}
```

Differences between handling binary & text data

`fread()` and `fwrite()` work with **binary files**, while functions like `fprintf()` and `fscanf()` work with **text files**.

Feature	Text File Functions (fprintf(), fscanf())	Binary File Functions (fwrite(), fread())
Data Format	Stores data as human-readable text	Stores raw binary data (not human-readable)
Usage	Useful for general document storage (logs, configs, CSV files)	Best for complex data (images, databases, structured records)
Speed	Slower (requires parsing and formatting)	Faster (direct memory-to-file storage)
Portability	Can be easily viewed/edited with text editors	Cannot be directly edited; requires specific programs
Precision	Might alter data due to newline or encoding differences	Stores exact binary representation, preserving structure
Functions Used	fprintf(), fscanf(), fputs(), fgets()	fwrite(), fread(), fseek(), ftell()

Writing and Reading Text Files

```
#include <stdio.h>
```

```
int main() {  
    FILE *file = fopen("data.txt", "w");  
    fprintf(file, "ID: %d, Name: %s, Salary: %.2f\n", 101, "Alice", 50000.5);  
    fclose(file);  
  
    file = fopen("data.txt", "r");  
    int id;  
    char name[20];  
    float salary;  
    fscanf(file, "ID: %d, Name: %s, Salary: %f", &id, name, &salary);  
    fclose(file);  
  
    printf("Read from text file: ID=%d, Name=%s, Salary=%.2f\n", id, name, salary);  
    return 0;  
}
```

Data stored in human-readable format

Requires **formatting** during input/output

Writing and Reading Binary Files

No formatting required

Stores **raw memory representation**

```
#include <stdio.h>
struct Employee {
    int id;
    char name[20];
    float salary;
};
int main() {
    FILE *file = fopen("data.bin", "wb");
    struct Employee emp = {101, "Alice", 50000.5};
    fwrite(&emp, sizeof(struct Employee), 1, file);
    fclose(file);
    file = fopen("data.bin", "rb");
    struct Employee readEmp;
    fread(&readEmp, sizeof(struct Employee), 1, file);
    fclose(file);
    printf("Read from binary file: ID=%d, Name=%s, Salary=%.2f\n", readEmp.id, readEmp.name,
readEmp.salary);
    return 0;
}
```


File Positioning Functions

- `fseek(fp, offset, position)` – Moves file pointer
- `ftell(fp)` – Returns current position
- `rewind(fp)` – Moves pointer to the beginning

```
fseek(file, 10, SEEK_SET);
```

```
long position = ftell(file);
```

File Positioning Functions

- `fseek(fp, offset, position)` – Moves file pointer
- `ftell(fp)` – Returns current position
- `rewind(fp)` – Moves pointer to the beginning

`fseek(file, 10, SEEK_SET);` *moves the file pointer to a specific position in the file. *

`file` → The file pointer.

`offset` → The number of bytes to move from the given position.

`origin` → The reference point (`SEEK_SET`, `SEEK_CUR`, `SEEK_END`).

Constant	Meaning
<code>SEEK_SET</code>	Moves to the given offset from the beginning.
<code>SEEK_CUR</code>	Moves relative to the current position.
<code>SEEK_END</code>	Moves relative to the end of the file.

File Positioning Functions

```
long position = ftell(file);
```

The function `ftell(FILE *file)` returns the **current position** of the file pointer in bytes.

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *file = fopen("example.txt", "r");
```

```
    if (file == NULL) {
```

```
        printf("Error opening file.\n");
```

```
        return 1;
```

```
    }
```

```
    fseek(file, 10, SEEK_SET); // Move 10 bytes from the beginning
```

```
    long position = ftell(file); // Get current file pointer location
```

```
    printf("Current file position: %ld\n", position);
```

```
    fclose(file);
```

```
    return 0;
```

```
}
```

Output (if example.txt exists and is non-empty)

Current file position: 10

Differences in File Positioning for Binary Files

No Interpretation of Special Characters

- In text files, newline characters (`\n`) may be treated differently across operating systems.
- Binary files store data exactly as it is, meaning no extra transformations occur.

Byte-Based Positioning

- `fseek()` moves the file pointer **by exact bytes** in binary files, without any adjustments.
- In text files, `fseek()` might behave unpredictably if dealing with multi-byte characters or newline conversions.

Reading and Writing Data in Blocks

- Binary files often use `fread()` and `fwrite()`, which handle structured data more efficiently.
- Positioning with `fseek()` in binary files allows skipping blocks of data, rather than character-by-character movement.

Differences in File Positioning for Binary Files

- **Use text files** for **simple** data storage where human readability is important.
- **Use binary files** for **structured** data storage that requires speed and precision.

Error Handling in File Operations

Check if the file pointer is NULL after `fopen()`

Use `perror()` for error messages

References

- ‘Let us C” by Yashawant Kanetkar
- <https://www.geeksforgeeks.org/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>